

FESA
Engenharia de Computação

Caio Alexandre Rossi – 081230013
Caio S. A. de Araújo - 081230014

MODULAÇÃO EM VHDL
PROJETOS NRZ-L E NRZ-I

Caio Alexandre Rossi – 081230013

Caio S. A. de Araújo - 081230014

**MODULAÇÃO EM VHDL
PROJETOS NRZ-L E NRZ-I**

Trabalho prático apresentado à disciplina de
Comunicação de Dados — Sistemas
Reconfiguráveis

Professor: Vinicius S. Borges

São Bernardo do Campo

2026

SUMÁRIO

1 OBJETIVO

2 ROTEIRO

2.1 Criação do projeto no Vivado

2.2 Adição dos arquivos VHDL

2.2.1 Código-fonte do circuito NRZ-L

2.2.2 Testbench do circuito NRZ-L

2.2.3 Código-fonte do circuito NRZ-I

2.2.4 Testbench do circuito NRZ-I

2.3 Configuração e execução da simulação

2.4 Parâmetros utilizados

3 RESULTADOS

3.1 Modulação Unipolar NRZ-L

3.2 Modulação Unipolar NRZ-I

4 ANÁLISE

4.1 Comportamento observado

4.2 Comparação entre NRZ-L e NRZ-I

4.3 Interpretação dos resultados

5 CONCLUSÃO

1 OBJETIVO

Este trabalho tem como objetivo desenvolver e simular, em VHDL utilizando o software Vivado, dois sistemas de modulação digital, Unipolar NRZ-L e Unipolar NRZ-I. Para cada modulação, foi implementado um circuito principal (source) e um testbench independente, sendo o comportamento dos sinais analisado por meio das formas de onda geradas no Vivado Simulator.

2 ROTEIRO

Esta seção descreve o passo a passo seguido para o desenvolvimento dos dois projetos. Como cada modulação foi implementada em um projeto Vivado independente, o procedimento descrito a seguir foi aplicado tanto para o NRZ-L quanto para o NRZ-I.

2.1 Criação do projeto no Vivado

Inicialmente foi aberto o software Vivado e selecionada a opção Create New Project. O nome do projeto foi definido como nrz_l_modulacao para o Projeto 1 e nrz_i_modulacao para o Projeto 2, escolhendo-se o diretório de destino. Foi selecionado o tipo RTL Project, marcando-se a opção "Do not specify sources at this time". Em seguida, o assistente foi concluído com Finish.

2.2 Adição dos arquivos VHDL

Cada projeto contém apenas dois arquivos VHDL, um arquivo de source com a implementação da modulação e um arquivo de testbench com a sequência de bits de entrada e a geração de clock para a simulação. O arquivo source foi adicionado por meio do menu Add Sources, opção Add or Create Design Sources, sendo posteriormente marcado como top com a opção Set as Top. O testbench foi adicionado por meio da opção Add or Create Simulation Sources.

Os códigos-fonte completos dos quatro arquivos VHDL utilizados são apresentados nos quadros a seguir, acompanhados das respectivas explicações.

2.2.1 Código-fonte do circuito NRZ-L

O Quadro 1 apresenta o código-fonte do arquivo nrz_l.vhd, responsável pela implementação da modulação NRZ-L.

Quadro 1 — Código-fonte do arquivo nrz_l.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity nrz_l is
    generic (
        N_BITS      : integer := 16;
        CLKS_PER_BIT : integer := 4
    );
    Port (
        clk      : in  STD_LOGIC;
        rst      : in  STD_LOGIC;
        data_in  : in  STD_LOGIC_VECTOR(N_BITS-1 downto 0);
        nrz_out  : out STD_LOGIC
    );
end nrz_l;

architecture Behavioral of nrz_l is
    signal bit_index      : integer range 0 to N_BITS-1 := 0;
    signal clk_counter    : integer range 0 to CLKS_PER_BIT-1 := 0;
begin
    process(clk, rst)
    begin
        if rst = '1' then
            bit_index <= 0;
            clk_counter <= 0;
        elsif rising_edge(clk) then
            if clk_counter = CLKS_PER_BIT-1 then
                clk_counter <= 0;
                if bit_index = N_BITS-1 then
                    bit_index <= 0;
                else
                    bit_index <= bit_index + 1;
                end if;
            else
                clk_counter <= clk_counter + 1;
            end if;
        end if;
    end process;

    nrz_out <= data_in(N_BITS-1 - bit_index);
end Behavioral;
```

O cabeçalho do arquivo importa a biblioteca IEEE.STD_LOGIC_1164, que fornece os tipos lógicos básicos utilizados em VHDL, como STD_LOGIC e STD_LOGIC_VECTOR. Em seguida, é declarada a entidade nrz_l, que define a interface do circuito.

A entidade utiliza dois parâmetros genéricos (generics): N_BITS, que define o tamanho do vetor de entrada (16 bits, equivalente ao número de switches da Basys 3), e CLKS_PER_BIT, que define quantos ciclos de clock representam cada bit transmitido (4 ciclos). O uso de generics permite alterar esses valores diretamente, sem necessidade de reescrever o código, atendendo ao requisito de parametrização do enunciado.

As portas declaradas são: clk (entrada de clock), rst (reset assíncrono), data_in (vetor de N_BITS bits com a sequência a ser transmitida) e nrz_out (saída modulada). A arquitetura Behavioral declara dois sinais internos: bit_index, que indica qual bit do vetor está sendo transmitido no instante atual, e clk_counter, que conta os ciclos de clock dentro de cada intervalo de bit.

O bloco process é sensível ao clock e ao reset. Quando rst está em nível alto, ambos os contadores são zerados de forma assíncrona. A cada borda de subida do clock, o clk_counter é incrementado; ao atingir CLKS_PER_BIT-1 (ou seja, ao final do intervalo do bit atual), o contador é zerado e o bit_index avança para o próximo bit. Ao chegar ao último bit (bit_index = N_BITS-1), o índice retorna a zero, fazendo com que a sequência se repita continuamente.

A última atribuição do código define a saída do circuito: nrz_out recebe diretamente o bit do vetor data_in correspondente ao bit_index atual. A expressão N_BITS-1 - bit_index garante a leitura do MSB (bit mais significativo) para o LSB (bit menos significativo). Como a saída é o próprio nível do bit, o circuito implementa exatamente a regra do NRZ-L: bit '1' resulta em nível alto e bit '0' em nível baixo, mantidos durante todo o intervalo de bit.

2.2.2 Testbench do circuito NRZ-L

O Quadro 2 apresenta o código do testbench tb_nrz_l.vhd, utilizado para validar o circuito NRZ-L por meio da simulação behavioral.

Quadro 2 — Código-fonte do arquivo tb_nrz_l.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity tb_nrz_l is
end tb_nrz_l;

architecture sim of tb_nrz_l is
    constant N_BITS      : integer := 16;
    constant CLKS_PER_BIT : integer := 4;
```

```

constant CLK_PERIOD    : time      := 10 ns;
constant BIT_PERIOD    : time      := 40 ns;

signal clk             : STD_LOGIC := '0';
signal rst             : STD_LOGIC := '1';
signal data_in         : STD_LOGIC_VECTOR(N_BITS-1 downto 0) :=
"1011001011010110";
signal nrz_out         : STD_LOGIC;

begin
  uut: entity work.nrz_1
    generic map (
      N_BITS          => N_BITS,
      CLKS_PER_BIT    => CLKS_PER_BIT
    )
    port map (
      clk             => clk,
      rst             => rst,
      data_in         => data_in,
      nrz_out         => nrz_out
    );

  clk_process: process
  begin
    clk <= '0';
    wait for CLK_PERIOD/2;
    clk <= '1';
    wait for CLK_PERIOD/2;
  end process;

  stim_process: process
  begin
    rst <= '1';
    wait for 20 ns;
    rst <= '0';
    wait for N_BITS * BIT_PERIOD + 50 ns;
    wait;
  end process;
end sim;

```

O testbench começa declarando uma entidade vazia (sem portas), padrão para arquivos de simulação em VHDL. Na arquitetura sim, são definidas constantes que parametrizam a simulação: N_BITS e CLKS_PER_BIT possuem os mesmos valores do circuito principal; CLK_PERIOD define o período do clock em 10 ns (correspondendo a 100 MHz); e BIT_PERIOD define a duração de cada bit em 40 ns. A constante BIT_PERIOD atende ao requisito do enunciado de que a duração do pulso seja facilmente modificável no testbench.

Em seguida, são declarados os sinais que conectam o testbench ao circuito sob teste: clk e rst são inicializados, e data_in recebe a sequência de teste "1011001011010110". Essa sequência foi escolhida por apresentar variações

suficientes (transições alternadas e repetições) para evidenciar o comportamento das duas modulações.

O componente é instanciado com o rótulo uut (unit under test) por meio das construções generic map e port map, que conectam os parâmetros e os sinais externos aos pinos da entidade nrz_l. O processo clk_process gera o clock continuamente, alternando entre 0 e 1 a cada metade do período. O processo stim_process aplica os estímulos: mantém o reset ativo por 20 ns, libera o reset, aguarda tempo suficiente para a transmissão completa dos 16 bits e, ao final, executa wait sem argumentos para encerrar a simulação.

2.2.3 Código-fonte do circuito NRZ-I

O Quadro 3 apresenta o código-fonte do arquivo nrz_i.vhd. A estrutura geral é semelhante à do circuito NRZ-L, mas com a adição de um sinal interno responsável por armazenar o nível anterior, característica fundamental da modulação NRZ-I.

Quadro 3 — Código-fonte do arquivo nrz_i.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity nrz_i is
  generic (
    N_BITS      : integer := 16;
    CLKS_PER_BIT : integer := 4
  );
  Port (
    clk      : in  STD_LOGIC;
    rst      : in  STD_LOGIC;
    data_in  : in  STD_LOGIC_VECTOR(N_BITS-1 downto 0);
    nrz_out  : out STD_LOGIC
  );
end nrz_i;

architecture Behavioral of nrz_i is
  signal bit_index   : integer range 0 to N_BITS-1 := 0;
  signal clk_counter : integer range 0 to CLKS_PER_BIT-1 := 0;
  signal level       : STD_LOGIC := '0';
begin
  process(clk, rst)
  begin
    if rst = '1' then
      bit_index   <= 0;
      clk_counter <= 0;
      level       <= '0';
    elsif rising_edge(clk) then
      if clk_counter = 0 then
        if data_in(N_BITS-1 - bit_index) = '1' then
          level <= not level;
        end if;
      end if;
    end if;
  end process;
end Behavioral;
```



```

        if clk_counter = CLKS_PER_BIT-1 then
            clk_counter <= 0;
            if bit_index = N_BITS-1 then
                bit_index <= 0;
            else
                bit_index <= bit_index + 1;
            end if;
        else
            clk_counter <= clk_counter + 1;
        end if;
    end if;
end process;

nrz_out <= level;
end Behavioral;

```

A interface da entidade `nrz_i` é idêntica à do `nrz_l`: mesmos generics (`N_BITS`, `CLKS_PER_BIT`) e mesmas portas (`clk`, `rst`, `data_in`, `nrz_out`). A diferença essencial está na arquitetura, que declara um sinal interno adicional chamado `level`, inicializado com '0'. Esse sinal armazena o nível atual da saída e é fundamental para o funcionamento da modulação NRZ-I, que depende do estado anterior para determinar o estado atual.

No bloco `process`, o reset zera não apenas os contadores, mas também o sinal `level`, garantindo que a simulação comece sempre em um estado conhecido. A diferença principal em relação ao NRZ-L está na lógica que ocorre no início de cada bit: quando `clk_counter` é igual a zero, o circuito verifica o valor do bit atual em `data_in`. Se o bit for '1', o sinal `level` é invertido por meio da operação `not level`; se for '0', o sinal é mantido inalterado. Esse comportamento implementa exatamente a regra do NRZ-I: bit '1' provoca transição e bit '0' mantém o nível anterior.

A lógica de avanço do `bit_index` e do `clk_counter` é idêntica à do NRZ-L. A última atribuição do código define que `nrz_out` recebe o valor de `level`, ou seja, a saída do circuito é o nível mantido pelo registrador interno, e não diretamente o bit de entrada. Essa indireção é o que dá ao NRZ-I sua propriedade de codificação por transição: o sinal de saída só muda quando há um bit '1' a ser transmitido, e a informação está justamente na presença ou ausência dessa transição.

2.2.4 Testbench do circuito NRZ-I

O Quadro 4 apresenta o código do testbench `tb_nrz_i.vhd`. A estrutura é equivalente à do testbench do NRZ-L, utilizando a mesma sequência de entrada para permitir comparação direta entre as duas modulações.

Quadro 4 — Código-fonte do arquivo `tb_nrz_i.vhd`

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity tb_nrz_i is
end tb_nrz_i;

architecture sim of tb_nrz_i is
    constant N_BITS          : integer := 16;
    constant CLKS_PER_BIT    : integer := 4;
    constant CLK_PERIOD      : time    := 10 ns;
    constant BIT_PERIOD      : time    := 40 ns;

    signal clk      : STD_LOGIC := '0';
    signal rst      : STD_LOGIC := '1';
    signal data_in  : STD_LOGIC_VECTOR(N_BITS-1 downto 0) :=
"1011001011010110";
    signal nrz_out  : STD_LOGIC;

begin
    uut: entity work.nrz_i
        generic map (
            N_BITS          => N_BITS,
            CLKS_PER_BIT    => CLKS_PER_BIT
        )
        port map (
            clk      => clk,
            rst      => rst,
            data_in  => data_in,
            nrz_out  => nrz_out
        );

    clk_process: process
    begin
        clk <= '0';
        wait for CLK_PERIOD/2;
        clk <= '1';
        wait for CLK_PERIOD/2;
    end process;

    stim_process: process
    begin
        rst <= '1';
        wait for 20 ns;
        rst <= '0';
        wait for N_BITS * BIT_PERIOD + 50 ns;
        wait;
    end process;
end sim;
```

O testbench `tb_nrz_i` possui exatamente a mesma estrutura do `tb_nrz_l` descrito anteriormente: as mesmas constantes de parametrização (`N_BITS`, `CLKS_PER_BIT`, `CLK_PERIOD`, `BIT_PERIOD`), os mesmos sinais de conexão (`clk`, `rst`, `data_in`, `nrz_out`) e a mesma sequência de entrada "1011001011010110". A única diferença é que a instância `uut` é mapeada para a entidade `nrz_i`, em vez de `nrz_l`.

A escolha de manter os parâmetros e a sequência de entrada idênticos nos dois testbenches é deliberada: dessa forma, as formas de onda obtidas nas duas simulações podem ser comparadas diretamente, evidenciando as diferenças de comportamento entre as duas modulações para a mesma entrada. O processo de geração de clock e de aplicação dos estímulos segue a mesma lógica descrita no testbench anterior.

2.3 Configuração e execução da simulação

No painel Flow Navigator, foi selecionada a opção `SIMULATION` e, em seguida, `Run Simulation`, `Run Behavioral Simulation`. Após a abertura do Vivado Simulator, o tempo de simulação foi ajustado para 1 μ s no campo "Run For" garantindo a visualização completa dos 16 bits da sequência de entrada. Os sinais internos relevantes (`bit_index` e, no caso do NRZ-I, `level`) foram adicionados à janela de waveform a partir do painel Scope/Objects, arrastando-os do módulo `uut`. O radix do sinal `data_in` foi alterado para Binary, permitindo a visualização direta dos 16 bits da sequência transmitida. Por fim, foi aplicado Zoom Fit para ajustar a forma de onda à janela de visualização.

2.4 Parâmetros utilizados

Os parâmetros adotados em ambos os projetos são apresentados na Tabela 1, a seguir.

Tabela 1 — Parâmetros utilizados nos projetos

Parâmetro	Valor	Descrição
<code>N_BITS</code>	16	Quantidade de bits do vetor de entrada
<code>CLKS_PER_BIT</code>	4	Número de ciclos de clock por bit (parametrizável)

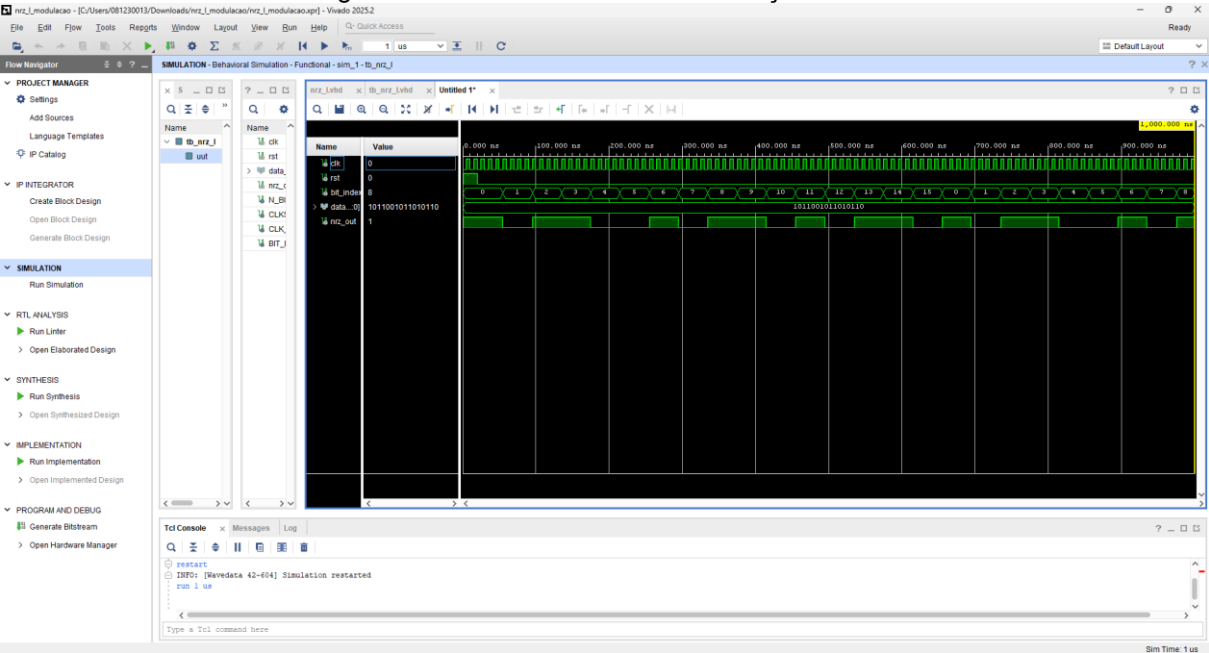
Parâmetro	Valor	Descrição
CLK_PERIOD	10 ns	Período do clock (100 MHz)
BIT_PERIOD	40 ns	Duração de cada bit ($4 \times$ CLK_PERIOD)
data_in	1011001011010110	Sequência de bits transmitida (MSB $\rightarrow$ LSB)

3 RESULTADOS

3.1 Modulação Unipolar NRZ-L

A Figura 1 apresenta a forma de onda obtida na simulação comportamental do circuito NRZ-L. São visíveis o clock, o sinal de reset, o índice do bit em transmissão, o vetor de entrada (data_in) em binário e a saída modulada (nrz_out).

Figura 1 — Forma de onda da modulação NRZ-L



A identificação dos sinais é descrita a seguir. O sinal clk corresponde ao clock de 100 MHz, com período de 10 ns. O sinal rst é o reset assíncrono, ativo em nível alto durante os primeiros 20 ns da simulação. O sinal bit_index indica o índice (de 0 a 15) do bit que está sendo transmitido em cada instante, permitindo mapear visualmente cada intervalo de bit. O sinal data_in representa o vetor de 16 bits em

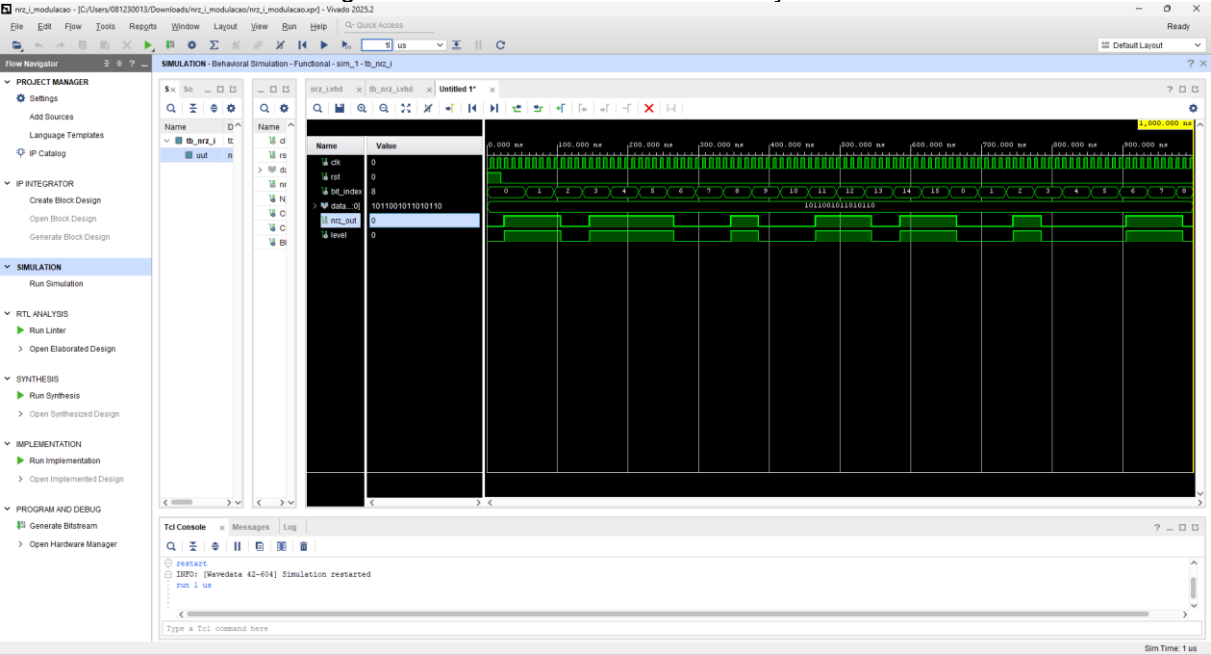
binário (1011001011010110), correspondendo à entrada de dados do sistema. Por fim, o sinal nrz_out é a saída modulada, na qual o nível alto representa o bit '1' e o nível baixo representa o bit '0', mantidos durante todo o intervalo de bit (40 ns).

O comportamento observado confirma a regra do NRZ-L: a saída é a representação direta do bit em transmissão, com cada nível mantido durante exatamente quatro ciclos de clock, totalizando 40 ns por bit.

3.2 Modulação Unipolar NRZ-I

A Figura 2 apresenta a forma de onda obtida na simulação behavioral do circuito NRZ-I, utilizando a mesma sequência de entrada do projeto NRZ-L para fins de comparação.

Figura 2 — Forma de onda da modulação NRZ-I



Os sinais clk, rst, bit_index e data_in possuem a mesma função descrita na seção 3.1. Adicionalmente, o sinal interno level armazena o nível de saída atual, sendo a fonte da saída nrz_out ($\text{nrz_out} \leq \text{level}$). Em NRZ-I, cada bit '1' provoca uma transição (inversão de nível) no início do intervalo de bit, e cada bit '0' mantém o nível anterior.

A Tabela 2 apresenta a sequência esperada da saída NRZ-I para a entrada utilizada, partindo de level = 0 após o reset.

Tabela 2 — Validação bit a bit da modulação NRZ-I

bit_index	data_in	Regra NRZ-I	nrz_out
0	1	inverte	1
1	0	mantém	1
2	1	inverte	0
3	1	inverte	1
4	0	mantém	1
5	0	mantém	1
6	1	inverte	0
7	0	mantém	0
8	1	inverte	1
9	1	inverte	0
10	0	mantém	0
11	1	inverte	1
12	0	mantém	1
13	1	inverte	0
14	1	inverte	1
15	0	mantém	1

A forma de onda obtida na simulação corresponde exatamente à sequência prevista na Tabela 2, validando a implementação do circuito NRZ-I.

4 ANÁLISE

4.1 Comportamento observado

Na modulação NRZ-L, observou-se uma correspondência direta entre o vetor de entrada e o sinal modulado: cada bit '1' resulta em um pulso de nível alto durante 40 ns, e cada bit '0' resulta em um pulso de nível baixo durante o mesmo intervalo.

Sequências consecutivas de bits iguais produzem patamares contínuos no sinal de saída.

Na modulação NRZ-I, o mesmo vetor de entrada produz uma forma de onda visualmente distinta. A saída não reflete diretamente o valor dos bits, mas sim suas transições: cada bit '1' provoca uma inversão do nível anterior, enquanto cada bit '0' preserva o nível

4.2 Comparação entre NRZ-L e NRZ-I

A comparação entre as duas modulações pode ser organizada em diferentes aspectos, conforme apresentado na Tabela 3.

Tabela 3 — Comparação entre as modulações NRZ-L e NRZ-I

	NRZ-L	NRZ-I
Representação do bit	Pelo nível absoluto (alto = 1, baixo = 0)	Pela presença ou ausência de transição no início do intervalo (1 = inverte, 0 = mantém)
Sequência longa de '1's	Nível alto contínuo (sem transições)	Transição a cada bit (favorece sincronização)
Sequência longa de '0's	Nível baixo contínuo (sem transições)	Nível mantido (sem transições)
Imunidade à inversão de polaridade	Baixa: a inversão dos fios troca a interpretação dos bits	Alta: a informação está nas transições, não no nível absoluto
Complexidade de implementação	Mínima (mapeamento direto bit → nível)	Maior (requer memorização do nível anterior)

4.3 Interpretação dos resultados

Os resultados obtidos evidenciam o principal compromisso entre as duas modulações. O NRZ-L é simples e legível diretamente, porém apresenta dificuldades em duas situações: sequências longas de bits iguais geram trechos sem transição, prejudicando a recuperação de clock no receptor; o esquema é sensível à inversão de

polaridade do meio de transmissão, pois a interpretação depende do nível absoluto do sinal.

O NRZ-I resolve parcialmente esses problemas. Como a informação é codificada nas transições, a inversão de polaridade não altera a decodificação. Além disso, sequências de bits '1' geram transições densas, ajudando o receptor a se sincronizar. No entanto, sequências longas de bits '0' continuam representando um problema, pois o nível permanece constante. Essa característica é facilmente verificada na simulação ao observar o intervalo correspondente aos bits '0' consecutivos da sequência de entrada, no qual o sinal `nrz_out` permanece estagnado.

Conclui-se que a escolha entre NRZ-L e NRZ-I depende das características do canal de comunicação e dos requisitos de sincronização do sistema. Em aplicações nas quais é importante garantir transições mesmo na presença de sequências de bits '1' (caso típico em comunicação assíncrona), o NRZ-I é preferível. Para canais bem sincronizados e quando a polaridade é garantida, o NRZ-L oferece a implementação mais simples e direta.

5 CONCLUSÃO

Os dois projetos foram implementados em VHDL e simulados com sucesso no Vivado Simulator. As formas de onda obtidas validam o funcionamento dos circuitos: a saída do NRZ-L corresponde diretamente ao vetor de bits transmitido, enquanto a saída do NRZ-I corresponde à sequência esperada gerada pela regra de inversão a cada bit '1'.